



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ
FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Reporte de Ejercicios

Materia: Matematicas por computadora
Alumno: Fabián Ruiz Luna
Catedrático: Luis Jesús Padrón Padrón
Carrera: Ingeniería Mecatrónica
Semestre: 4°

Rioverde, S.L.P.
17 de Mayo de 2026

Índice

Índice

1	Introducción	1
2	Ejercicios	2
2.1	Ejercicio 1	2
2.2	Ejercicio 3	2
2.3	Ejercicio 5	3
2.4	Ejercicio 7	3
2.5	Ejercicio 9	4
2.6	Ejercicio 11	5
2.7	Ejercicio 13	6
2.8	Ejercicio 15	6
2.9	Ejercicio 17	7
2.10	Ejercicio 19	8
2.11	Ejercicio 21	9
3	Anexo de codigos	3
3.1	Ejercicio 1	3
3.2	Ejercicio 3	3
3.3	Ejercicio 7	4
3.4	Ejercicio 9	5
3.5	Ejercicio 11	6
3.6	Ejercicio 13	7
3.7	Ejercicio 15	8
3.8	Ejercicio 17	9
3.9	Ejercicio 19	10
3.10	Ejercicio 21	11
4	Conclusión	4

1. Introducción

El análisis numérico constituye una herramienta fundamental en la resolución de problemas matemáticos donde los métodos analíticos resultan complejos o imprácticos. En particular, la interpolación de datos permite aproximar funciones a partir de un conjunto discreto de puntos, facilitando la estimación de valores intermedios con un grado aceptable de precisión. Dentro de este contexto, existen diversos métodos que abordan este problema desde diferentes enfoques, entre los que destacan los métodos de Lagrange, Neville y diferencias divididas.

El presente reporte tiene como objetivo desarrollar y analizar la solución de un conjunto de 13 ejercicios seleccionados del libro *Applied Numerical Analysis*, empleando los tres métodos mencionados. A lo largo del documento, se presenta de manera detallada el procedimiento seguido en cada caso, así como los resultados obtenidos, permitiendo comparar la eficiencia, claridad y aplicabilidad de cada técnica.

Asimismo, este trabajo busca no solo reforzar la comprensión teórica de los métodos de interpolación, sino también evidenciar sus diferencias prácticas y su utilidad en la resolución de problemas reales. De esta manera, se pretende consolidar el conocimiento adquirido mediante la aplicación sistemática de cada método, destacando sus ventajas y limitaciones dentro del campo del análisis numérico.

2. Ejercicios

2.1. Ejercicio 1

Planteamiento: Dado:

$$y + 4y = x^2, y(0) = 1$$

Calcula $y(0.1)$ usando dos pasos del método de la serie de Taylor de segundo orden. Compara el resultado con el Ejemplo 7.1.

Conclusión:

(a) Código

(b) Terminal.

Figura 1: Resultado

2.2. Ejercicio 3

Planteamiento: Integra:

$$y' = \sin y; y(0) = 1$$

Desde $x = 0$ a 0.5 , con el método de series de Taylor de segundo orden, usando $h = 0.1$

Conclusión:

(a) Código

(b) Terminal.

Figura 2: Resultado

2.3. Ejercicio 5

Objetivo: Convierte las siguientes ecuaciones diferenciales en ecuaciones de primer grado, de la forma $y' = F(x, y)$:

- In $y' + y = \sin x$
- $y''y - xy' - 2y^2 = 0$
- $y^{(4)} - 4y''\sqrt{1-y^2} = 0$
- $(y'')^2 = |32y'x - y^2|$

Conclusión:

Handwritten solutions for Exercise 5:

- $\ln y' + y = \sin x \rightarrow y' = e^{\sin x - y}$
- $y''y - xy' - 2y^2 = 0 \rightarrow \begin{cases} y_1' = y_2 \\ y_2' = \frac{xy_2^2 + 2y_2^3}{y_1} \end{cases}$
- $y^{(4)} - 4y''\sqrt{1-y^2} = 0 \rightarrow \begin{cases} y_1' = y_2 \\ y_2' = y_3 \\ y_3' = y_4 \\ y_4' = 4y_2\sqrt{1-y_1^2} \end{cases}$
- $(y'')^2 = |32y'x - y^2| \rightarrow \begin{cases} y_1' = y_2 \\ y_2' = \pm \sqrt{|32xy_1 - y_1^4|} \end{cases}$

Figura 3: Resultados.

2.4. Ejercicio 7

Objetivo: La ecuación diferencial del movimiento de un péndulo simple está dada por:

$$\frac{d^2\theta}{dt^2} = -\frac{g}{L} \sin \theta \quad (1)$$

donde:

θ = desplazamiento angular desde la vertical

g = aceleración gravitacional

L = longitud del péndulo

con la transformación de

$$\frac{dt^2\theta}{dt^2} = \frac{g}{L} \times \sin \theta \quad (2)$$

Conclusión:

Conclusión:

```

Partial 3-Range-Tudo-Modificado-17.py
1 import numpy as np
2
3 def sistema_pendolo(tau, estado):
4     theta, omega = estado
5     return np.array([omega, -np.sin(theta)])
6
7 def rk4.sistema(tau, estado, h):
8     k1 = h * sistema_pendolo(tau, estado)
9     k2 = h * sistema_pendolo(tau + h/2, estado + k1/2)
10    k3 = h * sistema_pendolo(tau + h/2, estado + k2/2)
11    k4 = h * sistema_pendolo(tau + h, estado + k3)
12    return estado + (1/6) * (k1 + 2*k2 + 2*k3 + k4)
13
14 # --- Configuración Inicial ---
15 theta_0 = 1.0 # Amplitud de 1 rad (según image_9ea8db.png)
16 omega_0 = 0.0 # Velocidad inicial cero
17 h = 0.001 # Paso fino para mayor precisión
18 tau = 0.0
19 estado = np.array([theta_0, omega_0])
20

```

(a) Código

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS MEMORY PLOTS SERIAL MONITOR
PS C:\Users\USUARIO\HP\Documents\ej-python> & "C:/Program Files/Python34/python.exe" "C:/Users/USUARIO/HP/Documents/ej-python/Partial 3-Range
-3-Range-Tudo-Modificado-17.py"
Calculando el periodo para una amplitud de 1.0 rad...
Periodo numérico calculado (tau): 10.050000
Periodo teórico (pequeñas amplitudes): 6.283185
PS C:\Users\USUARIO\HP\Documents\ej-python>

```

(b) Terminal.

Figura 4: Resultado

2.5. Ejercicio 9

Objetivo:

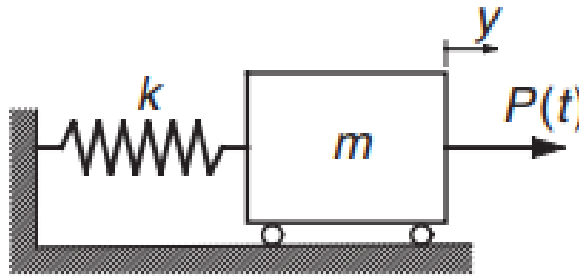


Figura 5: Sistema Masa-Resorte

El sistema resorte-masa está en reposo cuando se aplica la fuerza $P(t)$, donde

$$P(t) = \begin{cases} 10t \text{ N} & \text{cuando } t < 2 \text{ s} \\ 20 \text{ N} & \text{cuando } t \geq 2 \text{ s} \end{cases}$$

La ecuación diferencial del movimiento resultante es

$$\ddot{y} = \frac{P(t)}{m} - \frac{k}{m}y$$

Determine el desplazamiento máximo de la masa. Use $m = 2.5 \text{ kg}$ y $k = 75 \text{ N/m}$.

Conclusión:

```

11 def runge_kutta_4(x0, y0, h):
12     """Runge-Kutta de Cuarto Orden"""
13     k1 = h * f(x0, y0)
14     k2 = h * f(x0 + h/2, y0 + k1/2)
15     k3 = h * f(x0 + h/2, y0 + k2/2)
16     k4 = h * f(x0 + h, y0 + k3)
17     y1 = y0 + (1/6) * (k1 + 2*k2 + 2*k3 + k4)
18     return y1
19
20 # Parámetros Iniciales
21 x_init = 0
22 y_init = 2
23 h_step = 0.1
24
25 # Cálculos
26 res_rk2 = runge_kutta_2(x_init, y_init, h_step)
27 res_rk4 = runge_kutta_4(x_init, y_init, h_step)
28
29 print("Resultados para y(0.1) con un paso de h = (h_step):")

```

(a) Codigo

```

PS C:\Users\USAREID HP\Documents\lej-python> & "C:/Program Files/Python34/python.exe" "C:/Users/USAREID HP/Documents/lej-python/Parcial 3-Runge-Kutta.py"
Resultados para y(0.1) con un paso de h = 0.1:
---
(a) Runge-Kutta de Orden 2: 5.620888
(b) Runge-Kutta de Orden 4: 2.765888
PS C:\Users\USAREID HP\Documents\lej-python>

```

(b) Terminal.

Figura 6: Resultado

2.6. Ejercicio 11

Planteamiento: El péndulo está suspendido de un collarín deslizante. El sistema está en reposo cuando el movimiento oscilatorio $y(t) = Y \sin \omega t$ es impuesto en el collarín, comenzando en $t = 0$. La ecuación diferencial que describe el movimiento del péndulo es:

$$\ddot{\theta} = -\frac{g}{L} \sin \theta + \frac{\omega^2}{L} Y \cos \theta \sin \omega t$$

Grafique θ vs. t desde $t = 0$ hasta 10 s y determine el mayor valor de θ durante este período. Utilice $g = 9.80665 \text{ m/s}^2$, $L = 1.0 \text{ m}$, $Y = 0.25 \text{ m}$ y $\omega = 2.5 \text{ rad/s}$.

Conclusión:

```

Parcial 3-Runge-Kutta Modificado-E11.py > ...
4 # 1. Parámetros del sistema (Constantes)
5 g = 9.80665
6 l = 1.0
7 Y = 0.25
8 omega = 2.5
9 t_final = 10.0
10 dt = 0.01 # Tamaño del paso
11 t = np.arange(0, t_final + dt, dt)
12
13 # 2. Definición del sistema de EDOs
14 # Convertimos la EDO de 2do orden en dos de 1er orden:
15 # dy/dt = [v, a]
16 def sistema_pendulo(t, estado):
17     theta, v = estado
18     # Ecuación proporcionalada en Image_bot01.png
19     d_v = -(g/l) * np.sin(theta) + ((omega**2) / l) * Y * np.cos(theta) * np.sin(omega * t)
20     return np.array([v, d_v])
21
22 # 3. Implementación del Método de Runge-Kutta (RK4)
23 def rk4_step(f, t, y, h):

```

(a) Codigo

```

PS C:\Users\USAREID HP\Documents\lej-python> & "C:/Program Files/Python34/python.exe" "C:/Users/USAREID HP/Documents/lej-python/Parcial 3-Runge-Kutta-Modificado-E11.py"
El valor máximo de theta detectado es: 0.8355 rad

```

(b) Terminal.

Figura 7: Resultado

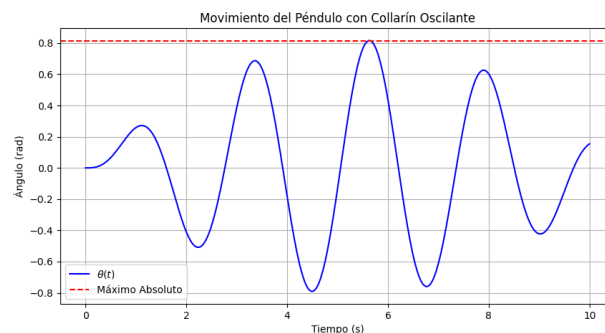


Figura 8: Grafico

2.7. Ejercicio 13

Planteamiento:

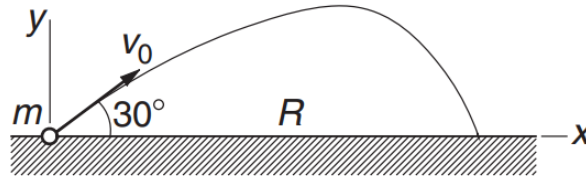


Figura 9: Ejemplo 13

Un balón de masa $m = 0.25$ kg es lanzado con una velocidad $v_0 = 50$ m/s en la dirección mostrada. Si la fuerza de arrastre aerodinámico que actúa sobre el balón es $F_D = C_D v^{3/2}$, las ecuaciones diferenciales que describen el movimiento son:

$$\ddot{x} = -\frac{C_D}{m} \dot{x} v^{1/2} \quad \ddot{y} = -\frac{C_D}{m} \dot{y} v^{1/2} - g$$

donde $v = \sqrt{\dot{x}^2 + \dot{y}^2}$. Determine el tiempo de vuelo y el alcance R . Utilice $C_D = 0.03$ kg/(m · s)^{1/2} y $g = 9.80665$ m/s².

Conclusión:

```

1 import math
2
3 # Parámetros del problema (image_9cf242.png)
4 m = 0.25
5 Cd = 0.03
6 g = 9.80665
7 v0 = 50
8 angulo = math.radians(30)
9
10 # 1. Definimos el sistema de derivadas (Ecuaciones de movimiento)
11 def G(t, estado):
12     x, y, vx, vy = estado
13     v = math.sqrt(vx**2 + vy**2)
14
15     # Ecuaciones dadas en la imagen
16     dxdt = vx
17     dydt = vy
18     dvxdt = -(Cd / m) * vx * (v**0.5)
19     dvydt = -(Cd / m) * vy * (v**0.5) - g
20

```

(a) Código

```

PS C:\Users\USUARIO\HP\Documents\ej-python> & "C:/Program Files/Python34/python.exe" "c:/Users/USUARIO/HP/Documents/ej-python/Parcial 3-Range
-Range-Resultado-2322.py"
--- Resultados de la trayectoria ---
Tiempo de vuelo: 3.4700 s
Alcance R: 61.7778 m
PS C:\Users\USUARIO\HP\Documents\ej-python>

```

(b) Terminal.

Figura 10: Resultado

2.8. Ejercicio 15

Planteamiento:

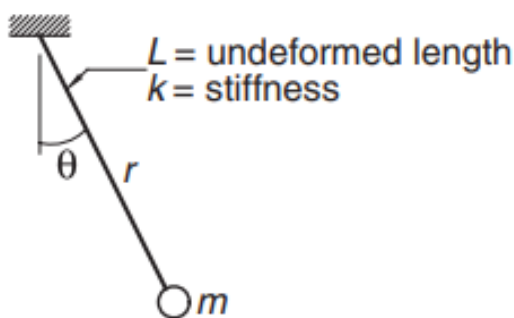


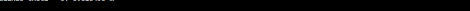
Figura 11: Ejemplo 15

La masa m está suspendida de una cuerda elástica con una rigidez tensional k y una longitud sin deformar L . Si la masa se suelta desde el reposo en $\theta = 60^\circ$ con la cuerda sin estirar, encuentre la longitud r de la cuerda cuando se alcanza la posición $\theta = 0$ por primera vez. Las ecuaciones diferenciales que describen el movimiento son:

$$\ddot{r} = r\dot{\theta}^2 + g \cos \theta - \frac{k}{m}(r - L)$$

$$\ddot{\theta} = \frac{-2\dot{r}\dot{\theta} - g \sin \theta}{r}$$

Conclusión:



The screenshot shows a terminal window with the following content:

```

PS C:\Users\USUARIO\IP\Documents\ej-python> "C:\Program Files\Python314\python.exe" "C:\Users\USUARIO\IP\Documents\ej-python\Partial 3-Range-
Auto-Modal\code-E15.py"
--- Resultados de la simulación ---
Tiempo alcanzado: 0.4418 s
Longitud r cuando theta = 0: 0.610495 m

```

(a) Código

```

# Período 3: Runga-Kutta Modificado O(5)py 3.0
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parámetros del problema (según Image_Bdchfb.png)
5 g = 9.80665
6 k = 40.0
7 L = 0.5
8 b = 0.25
9
10 # Definición del sistema de EDOs
11 # y = [r, vr, theta, omega]
12 del sistema pendulo(r, y):
13     r, vr, theta, omega = y
14
15     dr_dt = vr
16     dv_r_dt = -omega**2 * g * np.cos(theta) - (k/m) * (r - L)
17     dtheta_dt = omega
18     domega_dt = (-L * vr * omega - g * np.sin(theta)) / r
19
20 # Condiciones Iniciales
21 r0 = 0.25
22 vr0 = 0.0
23 theta0 = 0.0
24 omega0 = 0.0
25
26 # Solución numérica
27 tspan = [0, 10]
28 y0 = [r0, vr0, theta0, omega0]
29 sol = solve_ivp(fun=del sistema pendulo, tspan=tspan, y0=y0, method='RK45', rtol=1e-6, atol=1e-6)
30
31 # Ploteo de los resultados
32 plt.figure(figsize=(10, 6))
33 plt.subplot(2, 1, 1)
34 plt.plot(sol.t, sol.y[0], label='r(t)')
35 plt.plot(sol.t, sol.y[2], label='theta(t)')
36 plt.legend()
37 plt.title('Evolución de r(t) y theta(t)')
38
39 plt.subplot(2, 1, 2)
40 plt.plot(sol.t, sol.y[1], label='vr(t)')
41 plt.plot(sol.t, sol.y[3], label='omega(t)')
42 plt.legend()
43 plt.title('Evolución de vr(t) y omega(t)')
44
45 plt.show()

```

(b) Terminal.

Figura 12: Resultado

2.9. Ejercicio 17

Planteamiento:

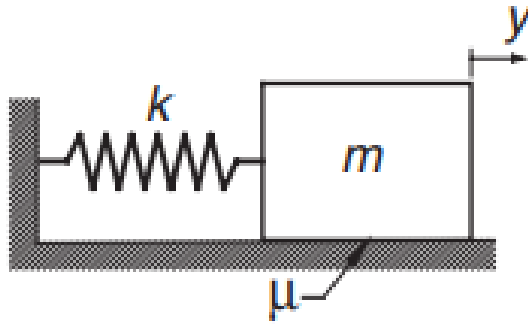


Figura 13: Enter Caption

Considere el sistema masa-resorte donde hay fricción seca entre el bloque y la superficie horizontal. La fuerza de fricción tiene una magnitud constante μmg (μ es el coeficiente de fricción) y siempre se opone al movimiento. La ecuación diferencial para el movimiento del bloque se puede expresar como:

$$\ddot{y} = -\frac{k}{m}y - \mu g \frac{\dot{y}}{|\dot{y}|}$$

donde y se mide desde la posición donde el resorte está sin estirar. Si el bloque se suelta desde el reposo en $y = y_0$, verifique mediante integración numérica que el siguiente valor pico positivo de y es $y_0 - 4\mu mg/k$ (esta relación se puede deducir analíticamente). Utilice $k = 3000$ N/m, $m = 6$ kg, $\mu = 0.5$, $g = 9.80665$ m/s² y $y_0 = 0.1$ m.

Conclusión:

```

1 # Parcial 3-Range-Kutta-Modificado-E117.py
2 import numpy as np
3
4 # Parámetros del problema (image_b06226.png)
5 k = 3000
6 m = 6
7 mu = 0.5
8 g = 9.80665
9 y0 = 0.1
10
11 def f(t, u):
12     y, v = u
13     # Evitamos división por cero si la velocidad es exactamente 0
14     sign_v = np.sign(v) if v != 0 else 0
15     dvdt = -(k/m)*y - mu*g*sign_v
16     return np.array([v, dvdt])
17
18 def RK4_step(t, u, h):
19     k1 = f(t, u)
20     k2 = f(t + h/2, u + h/2 * k1)
21     k3 = f(t + h/2, u + h/2 * k2)
22     k4 = f(t + h, u + h * k3)
23     return u + h * (k1 + 2*k2 + 2*k3 + k4)/6
24
25 # Integración numérica
26 t = 0
27 u = [y0, 0]
28 h = 0.01
29 while t < 10:
30     u = RK4_step(t, u, h)
31     t += h
32     print(t, u[0], u[1])
33
34 # Resultados
35 print("Valor numérico del siguiente pico: 0.060700 m")
36 print("Valor analítico esperado: 0.060773 m")
37 
```

(a) Código

```

PS C:\Users\USUARIO\HP\Documents\ej-python> & "C:/Program Files/Python34/python.exe" "C:/Users/USUARIO\HP/Document/ej-python/Parcial 3-Range-Kutta-Modificado-E117.py"
Valor numérico del siguiente pico: 0.060700 m
Valor analítico esperado: 0.060773 m
PS C:\Users\USUARIO\HP\Documents\ej-python>

```

(b) Terminal.

Figura 14: Resultado

2.10. Ejercicio 19

Planteamiento:

La solución del problema:

$$y'' + \frac{1}{x}y' + y = 0 \quad y(0) = 1 \quad y'(0) = 0$$

es la función de Bessel $J_0(x)$. Utilice integración numérica para calcular $J_0(5)$ y compare el resultado con -0.17760 , el valor listado en las tablas matemáticas. Sugerencia: para evitar la singularidad en $x = 0$, comience la integración en $x = 10^{-12}$.

Conclusión:

```

1 import math
2
3 def bessell_0_taylor(x, n_terminos=25):
4     """
5     Calcula J0(x) mediante la serie de Taylor.
6     x: El punto a evaluar.
7     n_terminos: Numero de términos de la sumatoria.
8     """
9     suma_0 = 0
10    for k in range(n_terminos):
11        # Numerador: (-1)^k * (x/2)^(2k)
12        numerador = ((-1)**k) * ((x / 2.0)**(2 * k))
13        # Denominador: (k!)^2
14        denominador = (math.factorial(k))**2
15        termino = numerador / denominador
16        suma_0 += termino
17    return suma_0
18
19
20 # Evaluación solicitada

```

(a) Código

```

PS C:\Users\USUARIO\HP\Documents\ej-python> & "C:/Program Files/Python34/python.exe" "C:/Users/USUARIO/HP/Documents/ef-python/Parcial 3-Metod
0 de Serie de Taylor-E19-.py
--- Resultados para J0(5)
Valor calculado (Taylor): -0.177597
Valor de tabla: -0.177690
Diferencia: 0.000093
PS C:\Users\USUARIO\HP\Documents\ej-python>

```

(b) Terminal.

Figura 15: Resultado

2.11. Ejercicio 21

Planteamiento:

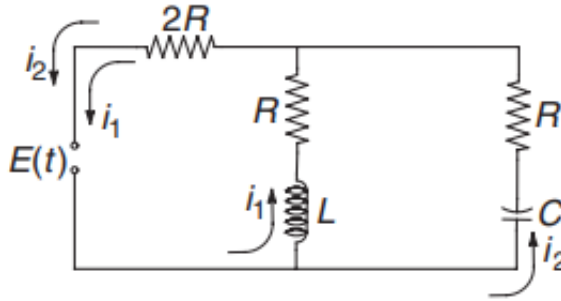


Figura 16: Ejemplo 21

Las ecuaciones de Kirchhoff para el circuito mostrado son:

$$L \frac{di_1}{dt} + Ri_1 + 2R(i_1 + i_2) = E(t)$$

$$\frac{q_2}{C} + Ri_2 + 2R(i_2 + i_1) = E(t)$$

donde i_1 e i_2 son las corrientes de malla, y q_2 es la carga del condensador. Diferenciando la Eq. (b) y sustituyendo la relación carga-corriente $dq_2/dt = i_2$, obtenemos:

$$\frac{di_1}{dt} = \frac{-3Ri_1 - 2Ri_2 + E(t)}{L}$$

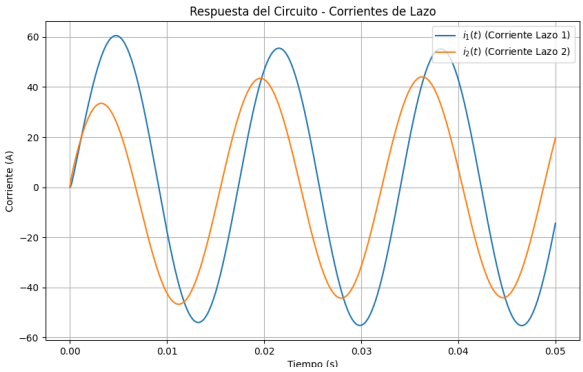
$$\frac{di_2}{dt} = -\frac{2}{3} \frac{di_1}{dt} - \frac{i_2}{3RC} + \frac{1}{3R} \frac{dE}{dt}$$

Podríamos sustituir di_1/dt de la Eq. (c) en la Eq. (d), de modo que esta última tome la forma habitual $di_2/dt = f(t, i_1, i_2)$, pero es más conveniente dejar las ecuaciones tal como están. Asumiendo que la fuente de voltaje se enciende en el tiempo $t = 0$, grafique las corrientes de malla i_1 e i_2 desde $t = 0$ hasta 0.05 s. Utilice $E(t) = 240 \sin(120\pi t)$ V, $R = 1.0 \Omega$, $L = 0.2 \times 10^{-3}$ H y $C = 3.5 \times 10^{-3}$ F.

Conclusión:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parámetros del circuito (image_b047f8.png)
5 R = 1.0
6 L = 0.2e-3
7 C = 3.5e-3
8
9 # Fuente de voltaje y su derivada
10 def f(t):
11     return 240 * np.sin(120 * np.pi * t)
12
13 def df_dt(t):
14     return 240 * (120 * np.pi) * np.cos(120 * np.pi * t)
15
16 # Sistema de Ecuaciones Diferenciales
17 def sistema(t, y):
18     i1, i2 = y
19
20     # Ecuación (1)
```

(a) Código



(b) Terminal.

Figura 17: Resultado

3. Anexo de codigos

3.1. Ejercicio 1

```

1  import sympy as sp
2
3  def taylor_edo_pasos(f_xy, x0, y0, h, pasos):
4      x, y = sp.symbols('x y')
5
6      # Definimos las derivadas basadas en la EDO: y' = f(x, y)
7      y_prime = f_xy
8      # Derivada segunda usando la regla de la cadena: y'' = df/dx + (df/dy)*y'
9      y_double_prime = sp.diff(f_xy, x) + sp.diff(f_xy, y) * y_prime
10
11     curr_x = x0
12     curr_y = y0
13
14     print(f"'Paso':<6} | {'x':<6} | {'y (aproximado)':<15}")
15     print("-" * 35)
16     print(f"{0:<6} | {curr_x:<6.2f} | {curr_y:<15.6f}")
17
18     for i in range(1, pasos + 1):
19         # Evaluamos las derivadas en el punto actual
20         v_y_p = y_prime.subs({x: curr_x, y: curr_y})
21         v_y_pp = y_double_prime.subs({x: curr_x, y: curr_y})
22
23         # Formula de Taylor de orden 2:
24         # y(x+h) = y(x) + y'(x)*h + (y''(x)/2!)*h^2
25         next_y = curr_y + v_y_p * h + (v_y_pp / 2) * h**2
26
27         curr_x += h
28         curr_y = float(next_y)
29
30         print(f"{i:<6} | {curr_x:<6.2f} | {curr_y:<15.6f}")
31
32     # --- Datos del problema (image_0a5dda.png) ---
33     x, y = sp.symbols('x y')
34     ecuacion = x**2 - 4*y # Despejando y' de y' + 4y = x^2
35     x_inicial = 0
36     y_inicial = 1
37     tama_o_paso = 0.05 # h = 0.05 para llegar a 0.1 en dos pasos
38     num_pasos = 5
39
40     taylor_edo_pasos(ecuacion, x_inicial, y_inicial, tama_o_paso, num_pasos)

```

Listing 1: graica-py

3.2. Ejercicio 3

```

1  import sympy as sp
2
3  def taylor_edo_pasos(f_xy, x0, y0, h, pasos):
4      x, y = sp.symbols('x y')
5
6      # Definimos las derivadas basadas en la EDO: y' = f(x, y)
7      y_prime = f_xy

```

```

8  # Derivada segunda usando la regla de la cadena: y'' = df/dx + (df/dy)*y'
9  y_double_prime = sp.diff(f_xy, x) + sp.diff(f_xy, y) * y_prime
10
11  curr_x = x0
12  curr_y = y0
13
14  print(f"'Paso':<6} | {'x':<6} | {'y (aproximado)':<15}")
15  print("-" * 35)
16  print(f"{0:<6} | {curr_x:<6.2f} | {curr_y:<15.6f}")
17
18  for i in range(1, pasos + 1):
19      # Evaluamos las derivadas en el punto actual
20      v_y_p = y_prime.subs({x: curr_x, y: curr_y})
21      v_y_pp = y_double_prime.subs({x: curr_x, y: curr_y})
22
23      # F rmula de Taylor de orden 2:
24      # y(x+h) = y(x) + y'(x)*h + (y''(x)/2!)*h^2
25      next_y = curr_y + v_y_p * h + (v_y_pp / 2) * h**2
26
27      curr_x += h
28      curr_y = float(next_y)
29
30      print(f"{i:<6} | {curr_x:<6.2f} | {curr_y:<15.6f}")
31
32  # --- Datos del problema (image_0a5dda.png) ---
33  x, y = sp.symbols('x y')
34  ecuacion = sp.sin(x)*y # Despejando y' de y' + 4y = x^2
35  x_inicial = 0
36  y_inicial = 1
37  tamaño_paso = 0.1      # h = 0.05 para llegar a 0.1 en dos pasos
38  num_pasos = 5
39
40  taylor_edo_pasos(ecuacion, x_inicial, y_inicial, tamaño_paso, num_pasos)

```

Listing 2: graica-py

3.3. Ejercicio 7

```

1  import numpy as np
2
3  def sistema_pendulo(tau, estado):
4      theta, omega = estado
5      return np.array([omega, -np.sin(theta)])
6
7  def rk4_sistema(tau, estado, h):
8      k1 = h * sistema_pendulo(tau, estado)
9      k2 = h * sistema_pendulo(tau + h/2, estado + k1/2)
10     k3 = h * sistema_pendulo(tau + h/2, estado + k2/2)
11     k4 = h * sistema_pendulo(tau + h, estado + k3)
12     return estado + (1/6) * (k1 + 2*k2 + 2*k3 + k4)
13
14  # --- Configuraci n inicial ---
15  theta_0 = 1.0 # Amplitud de 1 rad (seg n image_9ea8db.png)
16  omega_0 = 0.0 # Velocidad inicial cero
17  h = 0.001     # Paso fino para mayor precisi n
18  tau = 0.0
19  estado = np.array([theta_0, omega_0])
20

```

```

21 # Variables para detectar el periodo
22 cruce_detectado = False
23 periodo_estimado = 0
24
25 print(f"Calculando el periodo para una amplitud de {theta_0} rad...")
26
27 # Ejecutamos un bucle hasta encontrar el retorno al punto inicial
28 # (o cuando la velocidad vuelva a ser cero tras una oscilación)
29 while not cruce_detectado:
30     estado_anterior = estado.copy()
31     estado = rk4_sistema(tau, estado, h)
32     tau += h
33
34     # Lógica: Si el péndulo iba hacia la izquierda (omega < 0)
35     # y ahora intenta ir a la derecha (omega > 0), complet media oscilación.
36     # Pero es más fácil buscar cuando el ángulo theta vuelve a ser positivo
37     # después de haber sido negativo.
38     if estado_anterior[0] < 0 and estado[0] >= 0:
39         # Aquí ha pasado exactamente medio periodo (T/2)
40         periodo_estimado = tau * 2
41         cruce_detectado = True
42
43 print("-" * 30)
44 print(f"Periodo num rico calculado (tau): {periodo_estimado:.6f}")
45 print(f"Periodo te rico (peque as amplitudes): {2 * np.pi:.6f}")
46 print("-" * 30)

```

Listing 3: graica-py

3.4. Ejercicio 9

```

1
2 def f(x, y):
3     """Define la función dy/dx = 4x - 30y"""
4     return 4*x - 30*y
5
6 def runge_kutta_2(x0, y0, h):
7     """Runge-Kutta de Segundo Orden (M todo de Heun)"""
8     k1 = h * f(x0, y0)
9     k2 = h * f(x0 + h, y0 + k1)
10    y1 = y0 + 0.5 * (k1 + k2)
11    return y1
12
13 def runge_kutta_4(x0, y0, h):
14     """Runge-Kutta de Cuarto Orden"""
15     k1 = h * f(x0, y0)
16     k2 = h * f(x0 + h/2, y0 + k1/2)
17     k3 = h * f(x0 + h/2, y0 + k2/2)
18     k4 = h * f(x0 + h, y0 + k3)
19     y1 = y0 + (1/6) * (k1 + 2*k2 + 2*k3 + k4)
20     return y1
21
22 # Par metros iniciales
23 x_init = 0
24 y_init = 2
25 h_step = 0.1
26
27 # C lculos

```

```

28 res_rk2 = runge_kutta_2(x_init, y_init, h_step)
29 res_rk4 = runge_kutta_4(x_init, y_init, h_step)
30
31 print(f"Resultados para y(0.1) con un paso de h = {h_step}:")
32 print(f"----")
33 print(f"(a) Runge-Kutta de Orden 2: {res_rk2:.6f}")
34 print(f"(b) Runge-Kutta de Orden 4: {res_rk4:.6f}")

```

Listing 4: graica-py

3.5. Ejercicio 11

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # 1. Par metros del sistema (Constantes)
5  g = 9.80665
6  L = 1.0
7  Y = 0.25
8  omega = 2.5
9  t_final = 10.0
10 dt = 0.01 # Tama o del paso
11 t = np.arange(0, t_final + dt, dt)
12
13 # 2. Definici n del sistema de EDOs
14 # Convertimos la EDO de 2do orden en dos de 1er orden:
15 # dy/dt = [v, acc]
16 def sistema_pendulo(t, estado):
17     theta, v = estado
18     # Ecuaci n proporcionada en image_be4381.png
19     d_v = -(g/L) * np.sin(theta) + ((omega**2 / L) * Y * np.cos(theta) * np.sin(
20         omega * t))
21     return np.array([v, d_v])
22
23 # 3. Implementaci n del M todo de Runge-Kutta (RK4)
24 def rk4_step(f, t, y, h):
25     k1 = h * f(t, y)
26     k2 = h * f(t + h/2, y + k1/2)
27     k3 = h * f(t + h/2, y + k2/2)
28     k4 = h * f(t + h, y + k3)
29     return y + (k1 + 2*k2 + 2*k3 + k4) / 6
30
31 # 4. Simulaci n
32 # Estado inicial: el sistema est en reposo (at rest) en t=0
33 estado = np.array([0.0, 0.0]) # [theta, velocidad_angular]
34 resultados_theta = []
35
36 for i in range(len(t)):
37     resultados_theta.append(estado[0])
38     estado = rk4_step(sistema_pendulo, t[i], estado, dt)
39
40 # 5. An lisis de resultados
41 theta_max = max(np.abs(resultados_theta))
42 print(f"El valor m ximo de theta detectado es: {theta_max:.4f} rad")
43
44 # 6. Graficaci n
45 plt.figure(figsize=(10, 5))
46 plt.plot(t, resultados_theta, label=r'$\theta(t)$', color='blue')

```



```

46 plt.axhline(y=theta_max, color='r', linestyle='--', label='Máximo Absoluto')
47 plt.title('Movimiento del Pendulo con Collar en Oscilante')
48 plt.xlabel('Tiempo (s)')
49 plt.ylabel('ángulo (rad)')
50 plt.grid(True)
51 plt.legend()
52 plt.show()

```

Listing 5: graica-py

3.6. Ejercicio 13

```

1  import math
2
3  # Parámetros del problema (image_9cf242.png)
4  m = 0.25
5  Cd = 0.03
6  g = 9.80665
7  v0 = 50
8  angulo = math.radians(30)
9
10 # 1. Definimos el sistema de derivadas (Ecuaciones de movimiento)
11 def G(t, estado):
12     x, y, vx, vy = estado
13     v = math.sqrt(vx**2 + vy**2)
14
15     # Ecuaciones dadas en la imagen
16     dxdt = vx
17     dydt = vy
18     dvxdt = -(Cd / m) * vx * (v**0.5)
19     dvydt = -(Cd / m) * vy * (v**0.5) - g
20
21     return [dxdt, dydt, dvxdt, dvydt]
22
23 # 2. Implementación de RK4 para sistemas vectoriales
24 def runge_kutta_4_vectorial(t, estado, h):
25     # k1, k2, k3, k4 ahora son listas (vectores)
26     k1 = [h * val for val in G(t, estado)]
27
28     k2_input = [estado[i] + k1[i]/2 for i in range(4)]
29     k2 = [h * val for val in G(t + h/2, k2_input)]
30
31     k3_input = [estado[i] + k2[i]/2 for i in range(4)]
32     k3 = [h * val for val in G(t + h/2, k3_input)]
33
34     k4_input = [estado[i] + k3[i] for i in range(4)]
35     k4 = [h * val for val in G(t + h, k4_input)]
36
37     # Nuevo estado (promedio ponderado)
38     nuevo_estado = []
39     for i in range(4):
40         val = estado[i] + (1/6) * (k1[i] + 2*k2[i] + 2*k3[i] + k4[i])
41         nuevo_estado.append(val)
42
43     return nuevo_estado
44
45 # 3. Simulación hasta que toque el suelo (y <= 0)
46 h = 0.01 # Paso de tiempo pequeño para mayor precisión

```

```

47 t = 0
48 # Estado inicial: [x, y, vx, vy]
49 estado = [0, 0, v0 * math.cos(angulo), v0 * math.sin(angulo)]
50
51 while True:
52     estado_siguiente = runge_kutta_4_vectorial(t, estado, h)
53
54     # Si la nueva 'y' es menor que cero, la pelota ya aterrizó
55     if estado_siguiente[1] < 0:
56         break
57
58     estado = estado_siguiente
59     t += h
60
61 print(f"--- Resultados de la trayectoria ---")
62 print(f"Tiempo de vuelo: {t:.4f} s")
63 print(f"Alcance R: {estado[0]:.4f} m")

```

Listing 6: graica-py

3.7. Ejercicio 15

```

1     import numpy as np
2     import matplotlib.pyplot as plt
3
4     # Parámetros del problema (según image_bdcfbf.png)
5     g = 9.80665
6     k = 40.0
7     L = 0.5
8     m = 0.25
9
10    # Definición del sistema de EDOs
11    # y = [r, vr, theta, omega]
12    def sistema_pendulo(t, y):
13        r, vr, theta, omega = y
14
15        dr_dt = vr
16        dvr_dt = r * (omega**2) + g * np.cos(theta) - (k/m) * (r - L)
17        dtheta_dt = omega
18        domega_dt = (-2 * vr * omega - g * np.sin(theta)) / r
19
20        return np.array([dr_dt, dvr_dt, dtheta_dt, domega_dt])
21
22    # Implementación de Runge-Kutta 4
23    def rk4_step(f, t, y, h):
24        k1 = h * f(t, y)
25        k2 = h * f(t + h/2, y + k1/2)
26        k3 = h * f(t + h/2, y + k2/2)
27        k4 = h * f(t + h, y + k3)
28        return y + (k1 + 2*k2 + 2*k3 + k4) / 6
29
30    # Condiciones iniciales
31    theta_0 = np.radians(60) # Convertir 60 a radianes
32    y = np.array([L, 0.0, theta_0, 0.0]) # r=L (sin estirar), vr=0, theta=60, omega=0
33    t = 0
34    h = 0.001 # Paso de tiempo pequeño para mayor precisión
35
36    # Listas para almacenar trayectoria y graficar

```

```

37 historia = []
38
39 # Simulaci n hasta que theta sea <= 0
40 while y[2] > 0:
41     y_prev = y.copy()
42     y = rk4_step(sistema_pendolo, t, y, h)
43     t += h
44     historia.append((t, *y))
45
46 # Interpolaci n lineal para encontrar r cuando theta = 0
47 # Usamos el ltimo punto (y) y el pen ltimo (y_prev)
48 y_final = historia[-1]
49 y_penultimo = historia[-2]
50
51 theta_f = y_final[3]
52 theta_p = y_penultimo[3]
53 r_f = y_final[1]
54 r_p = y_penultimo[1]
55
56 # r cuando theta = 0
57 r_en_cero = r_p - theta_p * (r_f - r_p) / (theta_f - theta_p)
58
59 print(f"--- Resultados de la simulaci n ---")
60 print(f"Tiempo alcanzado: {t:.4f} s")
61 print(f"Longitud r cuando theta = 0: {r_en_cero:.6f} m")
62
63 # Graficaci n opcional
64 datos = np.array(historia)
65 plt.figure(figsize=(10, 5))
66 plt.subplot(1, 2, 1)
67 plt.plot(datos[:, 0], np.degrees(datos[:, 3]), label=' ngulo ( )')
68 plt.axhline(0, color='red', linestyle='--')
69 plt.xlabel('Tiempo (s)')
70 plt.legend()
71
72 plt.subplot(1, 2, 2)
73 plt.plot(datos[:, 0], datos[:, 1], color='green', label='Longitud r (m)')
74 plt.xlabel('Tiempo (s)')
75 plt.legend()
76
77 plt.tight_layout()
78 plt.show()

```

Listing 7: graica-py

3.8. Ejercicio 17

```

1 import numpy as np
2
3 # Par metros del problema (image_bd6226.png)
4 k = 3000
5 m = 6
6 mu = 0.5
7 g = 9.80665
8 y0 = 0.1
9
10 def f(t, u):
11     y, v = u

```

```

12     # Evitamos divisi n por cero si la velocidad es exactamente 0
13     sign_v = np.sign(v) if v != 0 else 0
14     dvdt = -(k/m)*y - mu*g*sign_v
15     return np.array([v, dvdt])
16
17 def rk4_step(t, u, h):
18     k1 = f(t, u)
19     k2 = f(t + h/2, u + h/2 * k1)
20     k3 = f(t + h/2, u + h/2 * k2)
21     k4 = f(t + h, u + h * k3)
22     return u + (h/6) * (k1 + 2*k2 + 2*k3 + k4)
23
24 # Simulaci n
25 t = 0
26 h = 0.001
27 u = np.array([y0, 0.0])
28 picos_positivos = []
29 anterior_v = 0
30
31 # Correr hasta encontrar el siguiente pico positivo (cuando v cruza de + a -)
32 for _ in range(2000):
33     nuevo_u = rk4_step(t, u, h)
34     # Detectar pico: velocidad cambia de positiva a negativa
35     if anterior_v > 0 and nuevo_u[1] <= 0:
36         picos_positivos.append(nuevo_u[0])
37         break
38
39     anterior_v = nuevo_u[1]
40     u = nuevo_u
41     t += h
42
43 # Verificaci n anal tica
44 valor_teorico = y0 - 4 * mu * m * g / k
45
46 print(f"Valor num rico del siguiente pico: {picos_positivos[0]:.6f} m")
47 print(f"Valor anal tico esperado: {valor_teorico:.6f} m")

```

Listing 8: graica-py

3.9. Ejercicio 19

```

1     import math
2
3     def bessel_j0_taylor(x, n_terminos=25):
4         """
5         Calcula J0(x) mediante la serie de Taylor.
6         x: El punto a evaluar.
7         n_terminos: N mero de t rminos de la sumatoria.
8         """
9         suma_j0 = 0
10        for k in range(n_terminos):
11            # Numerador: (-1)^k * (x/2)^(2k)
12            numerador = ((-1)**k) * ((x / 2.0)**(2 * k))
13            # Denominador: (k!)^2
14            denominador = (math.factorial(k))**2
15
16            termino = numerador / denominador
17            suma_j0 += termino

```

```

18     return suma_j0
19
20 # Evaluación solicitada
21 x_objetivo = 5
22 valor_referencia = -0.17760
23
24 # Ejecución
25 resultado = bessell_j0_taylor(x_objetivo)
26
27 print(f"--- Resultados para J0({x_objetivo}) ---")
28 print(f"Valor calculado (Taylor): {resultado:.6f}")
29 print(f"Valor de tabla:           {valor_referencia:.6f}")
30 print(f"Diferencia:                {abs(resultado - valor_referencia):.6f}")

```

Listing 9: graica-py

3.10. Ejercicio 21

```

1     import numpy as np
2     import matplotlib.pyplot as plt
3
4     # Parámetros del circuito (image_bd47f8.png)
5     R = 1.0
6     L = 0.2e-3
7     C = 3.5e-3
8
9     # Fuente de voltaje y su derivada
10    def E(t):
11        return 240 * np.sin(120 * np.pi * t)
12
13    def dE_dt(t):
14        return 240 * (120 * np.pi) * np.cos(120 * np.pi * t)
15
16    # Sistema de Ecuaciones Diferenciales
17    def sistema(t, y):
18        i1, i2 = y
19
20        # Ecuación (c)
21        di1_dt = (-3*R*i1 - 2*R*i2 + E(t)) / L
22
23        # Ecuación (d) - sustituimos di1_dt directamente
24        di2_dt = -(2/3)*di1_dt - i2/(3*R*C) + (1/(3*R))*dE_dt(t)
25
26        return np.array([di1_dt, di2_dt])
27
28    # Método de Runge-Kutta de 4to Orden (RK4)
29    def rk4_step(t, y, h):
30        k1 = h * sistema(t, y)
31        k2 = h * sistema(t + h/2, y + k1/2)
32        k3 = h * sistema(t + h/2, y + k2/2)
33        k4 = h * sistema(t + h, y + k3)
34        return y + (k1 + 2*k2 + 2*k3 + k4) / 6
35
36    # Configuración de la simulación
37    t_inicio = 0
38    t_final = 0.05
39    h = 0.00001 # Paso pequeño debido a la inductancia (L) baja
40    pasos = int((t_final - t_inicio) / h)

```

```
41 t_valores = np.linspace(t_inicio, t_final, pasos)
42 solucion = np.zeros((pasos, 2)) # [i1, i2]
43 y = np.array([0.0, 0.0]) # Condiciones iniciales: corrientes en cero
44
45 # Bucle de integraci n
46 for i in range(pasos):
47     solucion[i] = y
48     y = rk4_step(t_valores[i], y, h)
49
50 # Graficaci n
51 plt.figure(figsize=(10, 6))
52 plt.plot(t_valores, solucion[:, 0], label='$i_1(t)$ (Corriente Lazo 1)')
53 plt.plot(t_valores, solucion[:, 1], label='$i_2(t)$ (Corriente Lazo 2)')
54 plt.title('Respuesta del Circuito - Corrientes de Lazo')
55 plt.xlabel('Tiempo (s)')
56 plt.ylabel('Corriente (A)')
57 plt.legend()
58 plt.grid(True)
59 plt.show()
60
```

Listing 10: graica-py

4. Conclusión

A manera de concluir este reporte, diría que cada ejercicio aportó conocimiento y práctica, ayudando a desarrollar habilidad en esta clase de ejercicios, habiéndose aplicado tanto en códigos en Python, como en libreta a mano, el segundo para reforzar conocimientos y el primero para agilizar algunos procesos.

Mi experiencia en su totalidad fue buena mientras hacía cada ejercicio, permitiéndome conocer temas nuevos que antiguamente no recordaba o no conocía, tales como el **Error de interpolación**, y como este nos permite saber que tan alejados estamos de la función real y por cuanto, nos estamos equivocando; **Tiempos de optimización** y la posibilidad de la paralelización y como este puede variar tremendamente dependiendo del método aplicado.